0217. Contains Duplicate

y | Category: Arrays & Hashing | LeetCode: https://leetcode.com/problems/conta

Problem

- Given an integer array nums, return true if any value appears at least twice, otherwise false.
- Example: [1,2,3,1] → true, [1,2,3,4] → false.

Intuition

- We only need to know "have I seen this value before?" while scanning once.
- A HashSet answers that membership question in O(1) average time.

Brute Force

- Compare every pair (i, j) with i < j: O(n^2) time, O(1) space.
- Sorting first and checking neighbours is O(n log n) time and modifies the input.

Optimal Approach

- Create an empty HashSet.
- For each number: if set.add(x) returns false, x was already present, so return true.
- If the loop ends, every value was unique, so return false.

Worked Example

```
Input nums = [1, 2, 3, 1].

x=1  set={}         -> add -> {1}
x=2  set={1}        -> add -> {1,2}
x=3  set={1,2}      -> add -> {1,2,3}
x=1  set={1,2,3}    -> add fails -> return true
```

Java Solution

```java
public class ContainsDuplicate {
    public boolean containsDuplicate(int[] nums) {
        Set<Integer> seen = new HashSet<>();
        for (int x : nums) {
            if (!seen.add(x)) return true;
        }
        return false;
    }

    public boolean containsDuplicateSort(int[] nums) {
        int[] a = nums.clone();
        Arrays.sort(a);
        for (int i = 1; i < a.length; i++) if (a[i] == a[i - 1]) return true;
        return false;
    }
}
```

Complexity

- Time: O(n) average, one pass with O(1) set operations.
- Space: O(n) for the set in the worst case (all unique).

Pitfalls

- Use the boolean return of Set.add instead of contains + add (one hash lookup instead of two).
- Empty and single-element arrays are trivially false.